

Лабораторная работа №1

Зусь Александра, гр. 4404

Задача 1: Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Код программы:

```
using namespace std;
#include <iostream>
#include <omp.h>
int main(int argc, char* argv[]) {

    int num_threads;
    if (argc > 1) {
        num_threads = atoi(argv[1]);
    }

    omp_set_num_threads(num_threads);           // Устанавливаем количество потоков
    #pragma omp parallel                         // Параллельная область (здесь создаются потоки)
    {
        int id = omp_get_thread_num(); // Каждый поток получает свой id (0, 1, 2, ...)
        int nthreads = omp_get_num_threads(); // Общее количество потоков в этой параллельной области
        cout << "Hello World from thread " << id    // Печать (Порядок вывода не гарантирован)
              << " of " << nthreads << endl;
    }
    // После закрывающей скобки все потоки (кроме главного) завершаются

    return 0;
}
```

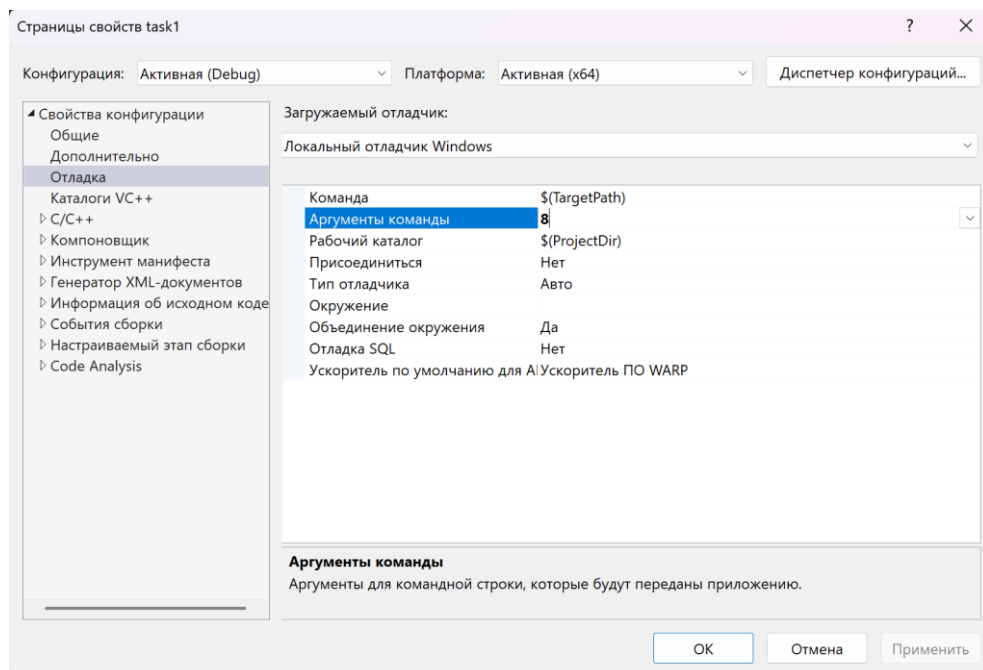
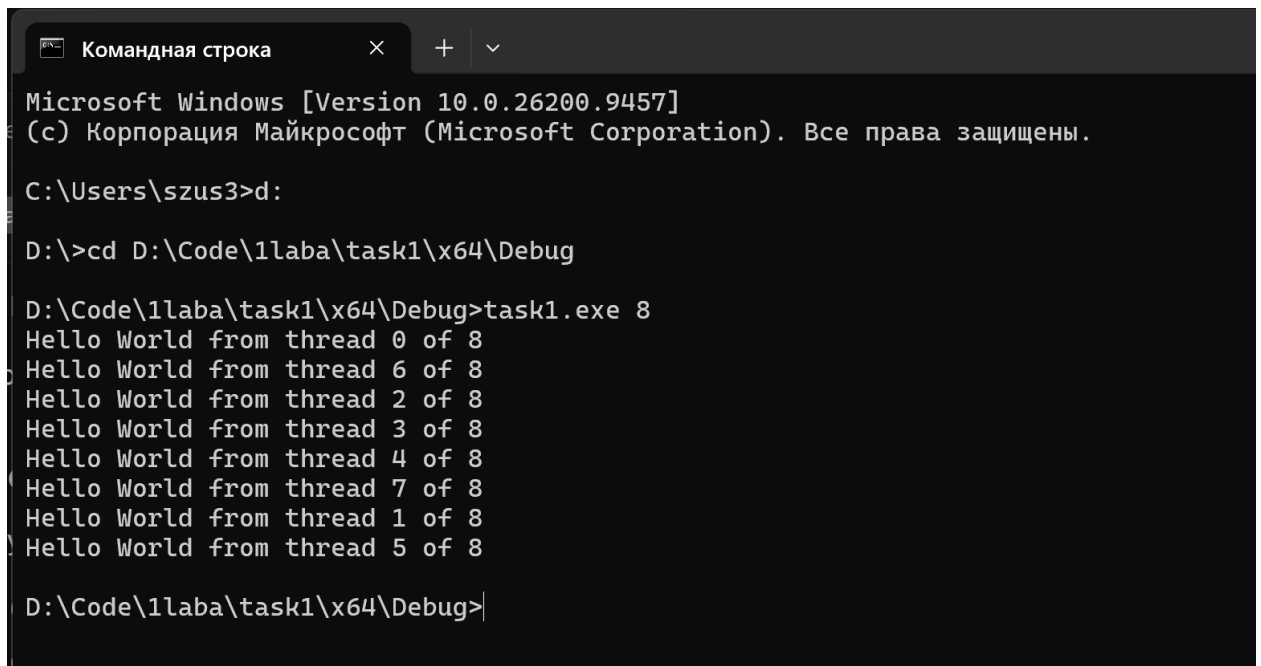


Рисунок 1. Значение аргумента - 8 (число потоков)

Рисунок 2. Задача №1, первый запуск

Рисунок 3. Задача №1, второй запуск

Рисунок 4. Задача №1, третий запуск



```
Командная строка
Microsoft Windows [Version 10.0.26200.9457]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\szus3>d:

D:\>cd D:\Code\11aba\task1\x64\Debug

D:\Code\11aba\task1\x64\Debug>task1.exe 8
Hello World from thread 0 of 8
Hello World from thread 6 of 8
Hello World from thread 2 of 8
Hello World from thread 3 of 8
Hello World from thread 4 of 8
Hello World from thread 7 of 8
Hello World from thread 1 of 8
Hello World from thread 5 of 8

D:\Code\11aba\task1\x64\Debug>
```

Рисунок 5. Задача №1, запуск через cmd

Ответ на вопрос: Вывод не всегда одинаковый. Строки появляются в разном порядке при каждом запуске. Это происходит потому, что:

- Все потоки выполняются параллельно (модель fork-join).
- Доступ к консоли (cout) не синхронизирован.
- Операционная система сама решает, в каком порядке потоки получают процессорное время.

Поэтому возникает классическая ошибка соревнования (race condition) на выводе. Согласно теории OpenMP, для устранения такой ситуации нужно использовать средства синхронизации (critical, ordered, barrier и т.д.).

Задача №2: Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1])/3.0$

Запустите программу с 8-ю процессами при различных типах распределения работ.

Код программы:

```
#include <iostream>
#include <omp.h>
#include <cstdlib>
#include <vector>
#include <iomanip>
using namespace std;

int main(int argc, char* argv[]) {
    if (argc < 3) {
        cerr << "Usage: " << argv[0] << " <num_threads> <array_size>\n";
        return 1;
    }

    int num_threads = atoi(argv[1]);
    int N = atoi(argv[2]); // размер массива (для задачи = 16000)

    if (num_threads <= 0 || N < 3) {
        cerr << "Bad arguments\n";
        return 1;
    }

    omp_set_num_threads(num_threads);

    // Создаём и инициализируем массивы
    vector<double> a(N);
    vector<double> b(N, 0.0); // сразу заполняем нулями

    // Инициализация a[i] = i (можно параллельно, но для простоты последовательно)
    for (int i = 0; i < N; ++i) {
        a[i] = static_cast<double>(i);
    }

    // Параллельный цикл с разными schedule (несколько запусков с заменой)
    #pragma omp parallel for schedule(static) // Вариант 1: static (по умолчанию)
    for (int i = 1; i < N - 1; ++i) {
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
    }

    // Печатаем несколько первых и последних элементов
    cout << "=== schedule(static) ===\n";
    cout << fixed << setprecision(2);
    for (int i = 0; i < 5; ++i)
        cout << "b[" << i << "] = " << b[i] << "\n";
    cout << "... \n";
    for (int i = N - 5; i < N; ++i)
        cout << "b[" << i << "] = " << b[i] << "\n";

    return 0;
}
```

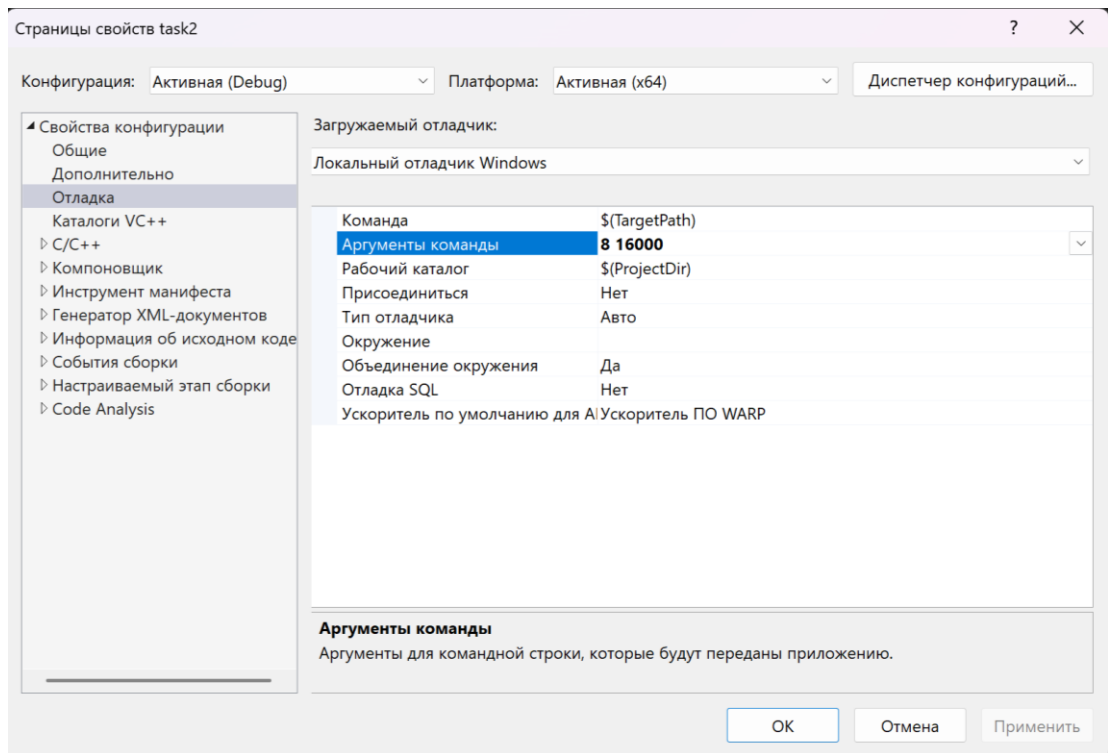


Рисунок 6. Значение аргумента - 8 16000 (число потоков и размер массива)

```

Консоль отладки Microsoft
=== schedule(static) ===
b[0] = 0.00
b[1] = 1.00
b[2] = 2.00
b[3] = 3.00
b[4] = 4.00
...
b[15995] = 15995.00
b[15996] = 15996.00
b[15997] = 15997.00
b[15998] = 15998.00
b[15999] = 0.00

D:\Code\11aba\task2\x64\Debug\task2.exe (процесс 24520) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:

```

Рисунок 7. Задача №2, static

Примечание: В коде меняю строку «#pragma omp parallel for schedule(static)» на «#pragma omp parallel for schedule(dynamic)»

```

Консоль отладки Microsoft
=== schedule(dynamic) ===
b[0] = 0.00
b[1] = 1.00
b[2] = 2.00
b[3] = 3.00
b[4] = 4.00
...
b[15995] = 15995.00
b[15996] = 15996.00
b[15997] = 15997.00
b[15998] = 15998.00
b[15999] = 0.00

D:\Code\11aba\task2\x64\Debug\task2.exe (процесс 8424) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:

```

Рисунок 8. Задача №2, dynamic

Примечание: В коде меняю строку «`#pragma omp parallel for schedule(dynamic)`» на «`#pragma omp parallel for schedule(guided)`»

```
Консоль отладки Microsoft
=== schedule(guided) ===
b[0] = 0.00
b[1] = 1.00
b[2] = 2.00
b[3] = 3.00
b[4] = 4.00
...
b[15995] = 15995.00
b[15996] = 15996.00
b[15997] = 15997.00
b[15998] = 15998.00
b[15999] = 0.00

D:\Code\1laba\task2\x64\Debug\task2.exe (процесс 16380) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 9. Задача №2, guided

```
Командная строка
Microsoft Windows [Version 10.0.26200.9457]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\szus3>d:

D:\>cd D:\Code\1laba\task2\x64\Debug

D:\Code\1laba\task2\x64\Debug>task2.exe 8 16000
=== schedule(guided) ===
b[0] = 0.00
b[1] = 1.00
b[2] = 2.00
b[3] = 3.00
b[4] = 4.00
...
b[15995] = 15995.00
b[15996] = 15996.00
b[15997] = 15997.00
b[15998] = 15998.00
b[15999] = 0.00

D:\Code\1laba\task2\x64\Debug>
```

Рисунок 10. Задача №2, guided запуск через cmd

Ответ на вопрос: Цикл не имеет зависимости по данным между итерациями (можно безопасно распараллеливать).

- `schedule(static)` — итерации заранее делятся между потоками равными кусками. Хорошо работает, когда время выполнения итераций примерно одинаковое.
- `schedule(dynamic)` — потоки берут порции по мере освобождения. Лучше балансирует нагрузку, но имеет чуть большие накладные расходы.
- `schedule(guided)` — размер порции уменьшается по ходу выполнения. Компромисс между `static` и `dynamic`.

Во всех трёх случаях результат вычисления `b[i]` одинаковый, меняется только способ раздачи работы потокам.

Задача №3: Модифицируйте задачу 1 так, что бы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

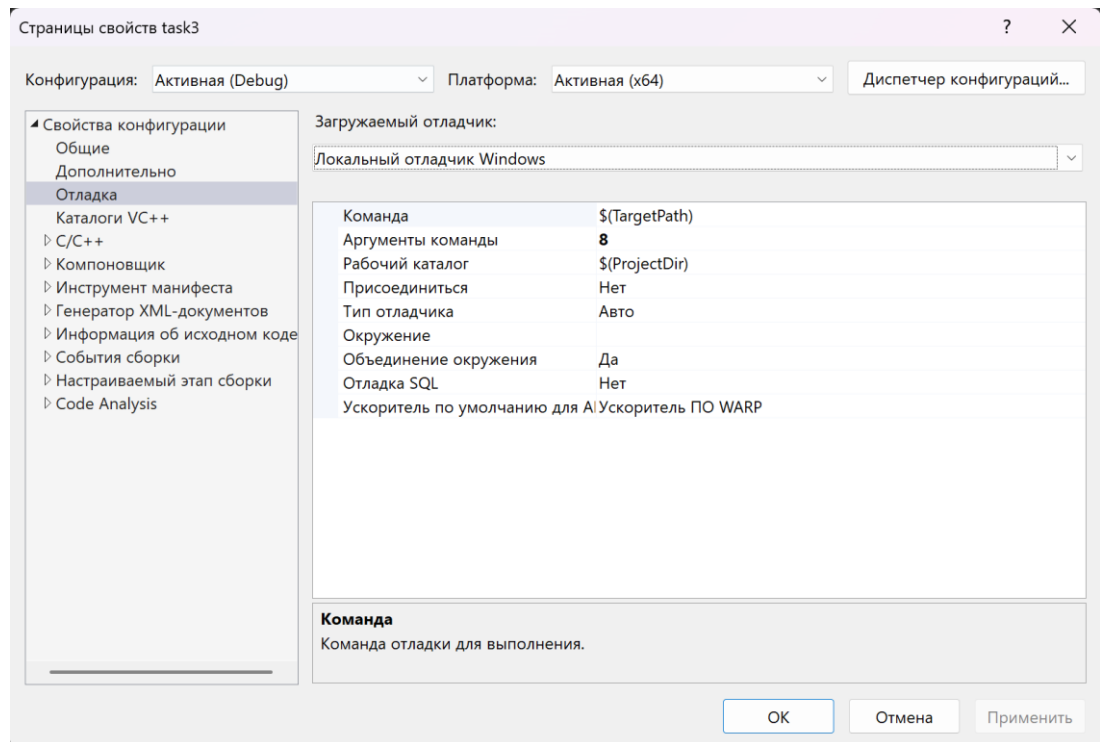


Рисунок 9. Значение аргумента - 8 (число потоков)

Способ 1. «Конструкция single»

Код программы:

```
#include <iostream>#include <omp.h>#include <cstdlib>using namespace std;
int main(int argc, char* argv[])
{
    if (argc < 2) {
        cerr << "Укажите число потоков\n";
        cerr << "Пример: " << argv[0] << " 8\n";
        return 1;
    }

    int num_threads = atoi(argv[1]);
    if (num_threads <= 0) {
        cerr << "Число потоков должно быть положительным\n";
        return 1;
    }

    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        int total = omp_get_num_threads();
        #pragma omp single
        {
            // Один поток печатает все номера в обратном порядке
            for (int id = total - 1; id >= 0; --id) {
                cout << "Thread " << id << " of " << total << ": Hello World\n";
            }
        }
    }
    return 0;
}
```

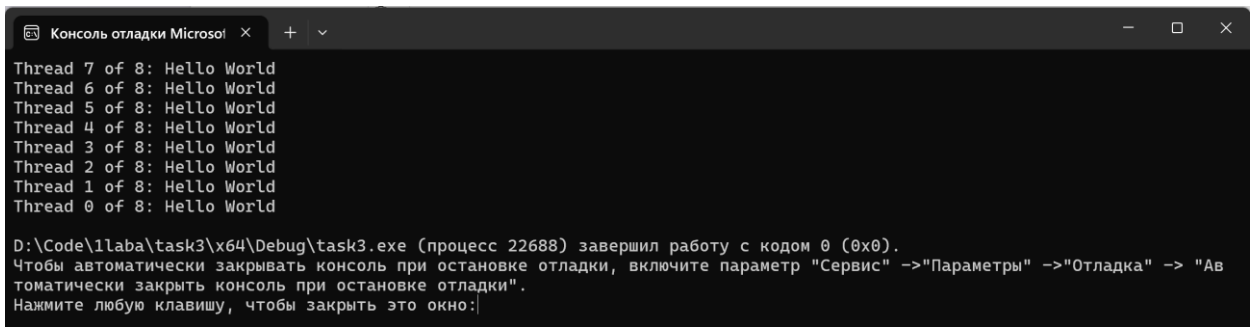


Рисунок 11. Задача №3, способ №1 - single

Примечание: Конструкция single гарантирует, что блок кода выполнит только один поток. Остальные потоки ждут его завершения.

Способ 2. «Критическая секция + общий счётчик»

Код программы:

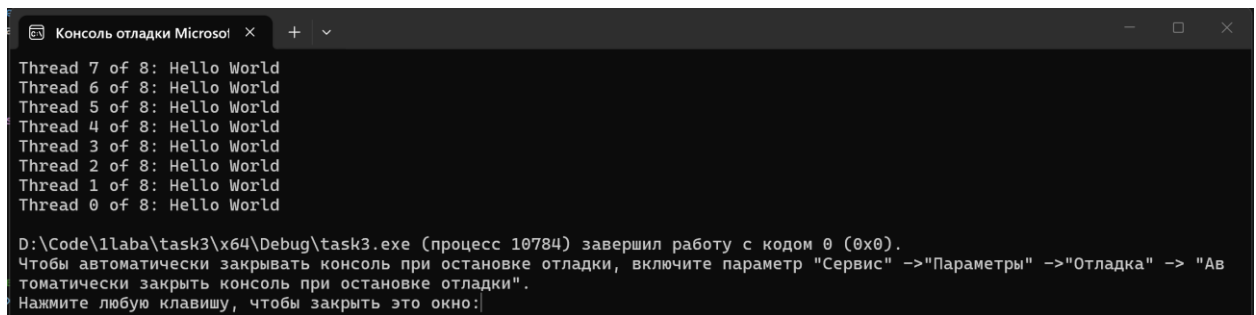
```
#include <iostream>
#include <omp.h>
#include <cstdlib>
using namespace std;
int main(int argc, char* argv[])
{
    if (argc < 2) {
        cerr << "Укажите число потоков\n";
        cerr << "Пример: " << argv[0] << " 8\n";
        return 1;
    }

    int num_threads = atoi(argv[1]);
    if (num_threads <= 0) {
        cerr << "Число потоков должно быть положительным\n";
        return 1;
    }

    omp_set_num_threads(num_threads);

    int next = 0; // общий счётчик (shared)
    #pragma omp parallel shared(next)
    {
        int my_id = omp_get_thread_num();
        int total = omp_get_num_threads();

        while (true) {#pragma omp critical
            {
                if (next >= total) {
                    // все уже напечатали — выходим
                }
                else {
                    int id_to_print = total - 1 - next;
                    if (my_id == id_to_print) {
                        cout << "Thread " << my_id << " of " << total << ": Hello World\n";
                        ++next;
                    }
                }
            }
            // Проверяем условие выхода вне critical
            if (next >= total) break;
        }
    }
    return 0;
}
```



```
Thread 7 of 8: Hello World
Thread 6 of 8: Hello World
Thread 5 of 8: Hello World
Thread 4 of 8: Hello World
Thread 3 of 8: Hello World
Thread 2 of 8: Hello World
Thread 1 of 8: Hello World
Thread 0 of 8: Hello World

D:\Code\11aba\task3\x64\Debug\task3.exe (процесс 10784) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 12. Задача №3, способ №2 - Критическая секция + общий счётчик

Примечание: Критическая секция (critical) обеспечивает взаимное исключение - только один поток может находиться внутри неё в один момент времени.

Способ 3. «Использование барьеров (barrier)»

Код программы:

```
#include <iostream>
#include <omp.h>
#include <cstdlib>
using namespace std;
int main(int argc, char* argv[])
{
    if (argc < 2) {
        cerr << "Укажите число потоков\n";
        cerr << "Пример: " << argv[0] << " 8\n";
        return 1;
    }

    int num_threads = atoi(argv[1]);
    if (num_threads <= 0) {
        cerr << "Число потоков должно быть положительным\n";
        return 1;
    }

    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        int my_id = omp_get_thread_num();
        int total = omp_get_num_threads();

        // Идём от большего номера к меньшему
        for (int id = total - 1; id >= 0; --id) {
            if (my_id == id) {
                cout << "Thread " << my_id << " of " << total << ": Hello World\n";
            }

            // Все потоки ждут, пока текущий id не напечатает
            #pragma omp barrier
        }
    }

    return 0;
}
```

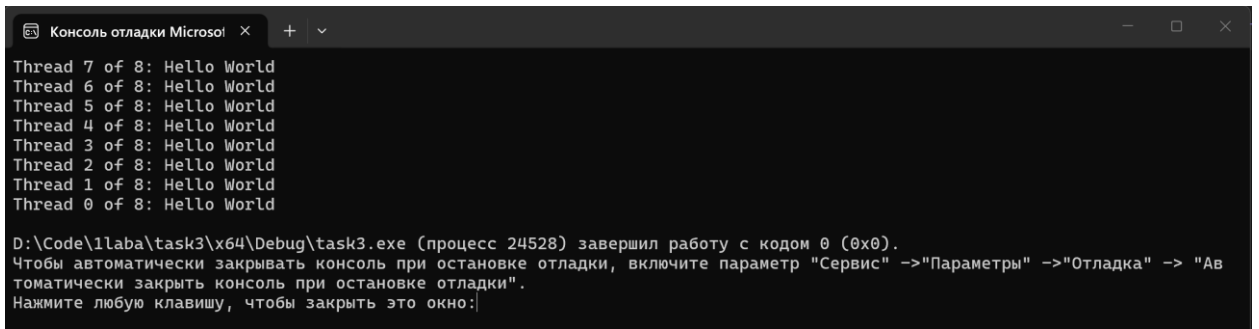


Рисунок 13. Задача №3, способ №3 - Использование барьеров (barrier)

Примечание: #pragma omp barrier - точка синхронизации. Все потоки ждут, пока последний из них не дойдёт до барьера.

Способ 4. «Директива master»

Код программы:

```
#include <iostream>
#include <omp.h>
#include <cstdlib>
using namespace std;
int main(int argc, char* argv[])
{
    if (argc < 2) {
        cerr << "Укажите число потоков\n";
        cerr << "Пример: " << argv[0] << " 8\n";
        return 1;
    }

    int num_threads = atoi(argv[1]);
    if (num_threads <= 0) {
        cerr << "Число потоков должно быть положительным\n";
        return 1;
    }

    omp_set_num_threads(num_threads);
#pragma omp parallel
    {
        int total = omp_get_num_threads();
#pragma omp master
        {
            // Только главный поток (номер 0) печатает все номера
            for (int id = total - 1; id >= 0; --id) {
                cout << "Thread " << id << " of " << total << ": Hello World\n";
            }
        }
    }

    return 0;
}
```

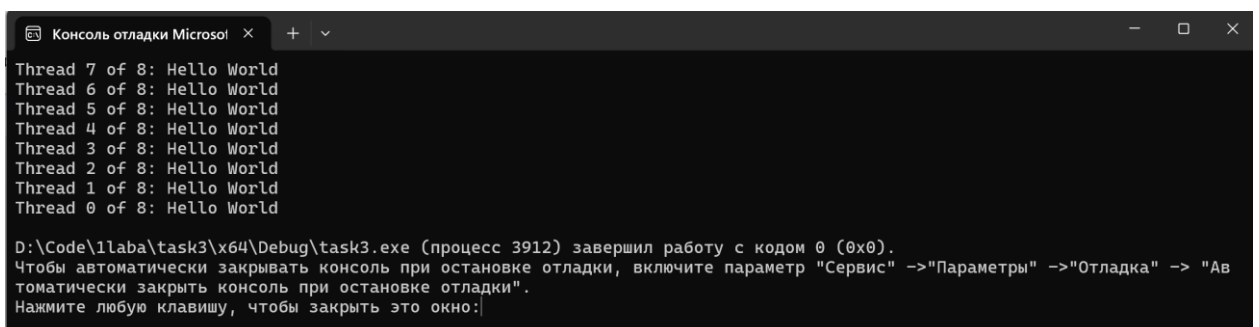
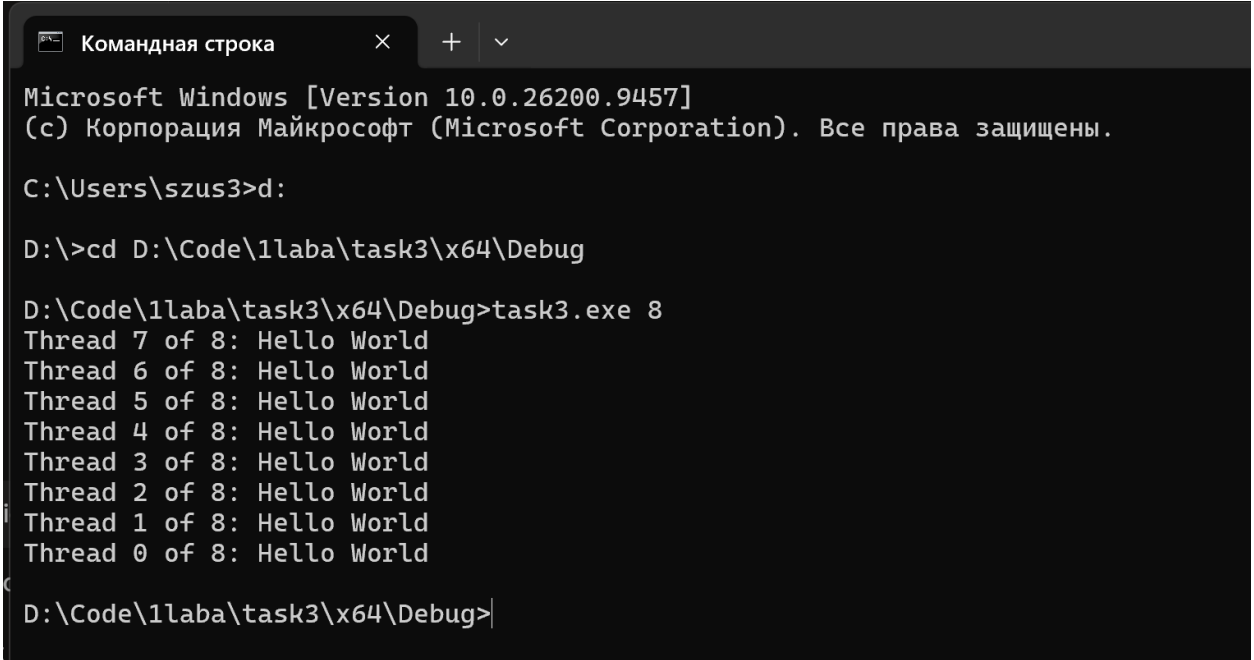


Рисунок 14. Задача №3, способ №4 - Директива master

Примечание: Директива master выполняет блок только основным потоком (потоком с номером 0). Неявного барьера в конце нет.



```
Командная строка
Microsoft Windows [Version 10.0.26200.9457]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\szus3>d:

D:\>cd D:\Code\11aba\task3\x64\Debug

D:\Code\11aba\task3\x64\Debug>task3.exe 8
Thread 7 of 8: Hello World
Thread 6 of 8: Hello World
Thread 5 of 8: Hello World
Thread 4 of 8: Hello World
Thread 3 of 8: Hello World
Thread 2 of 8: Hello World
Thread 1 of 8: Hello World
Thread 0 of 8: Hello World

D:\Code\11aba\task3\x64\Debug>
```

Рисунок 15. Задача №3, запуск через cmd